

System Understanding & Backend Reliability Brief

Why we are building this · why it is necessary · how it works ·
and what the requested backend overhaul means

Prepared for	Prepared by	Status
Founder, Shramik Sathi	Engineering (Claude)	Pre-implementation review

This document is my understanding of the product and the goals behind the requested backend work. No code or database changes have been made based on it yet — it is for your review and sign-off first.

1. Why we are building this

Shramik Sathi (“Worker’s Companion”) exists to connect India’s industrial employers with blue-collar / contract workers on a single trusted rail, while automating the heavy legal and financial compliance that sits between them. It is a two-sided marketplace plus a workforce-management and compliance system.

In plain terms: employers find verified workers and stay legally compliant; workers find real jobs, fair wages, and a portable digital identity.

2. Why it is necessary

Around 90% of India’s workforce is informal. For blue-collar and contract labour, two sides each carry real pain that no existing platform solves together:

For workers

- **No verified access to jobs** — they depend on middlemen/contractors, and fake listings and wage scams are common.
- **Wage theft & no benefits** — pay is often delayed or short, with no PF (provident fund) or ESI (health insurance).
- **No portable identity or work history** — a worker must re-prove who they are at every new site; nothing follows them.
- **Unsafe conditions** with no safe way to report problems.

For employers (industrial enterprises)

- **Heavy statutory compliance** under India’s labour laws (Contract Labour Act, Code on Wages, EPF, ESIC, registers, returns) — mistakes mean penalties.
- **Hard to source background-verified labour** at scale and on demand.
- **Manual, error-prone payroll, attendance, gate-passes and statutory registers.**

There is no single trusted platform that links *compliant employers* to *verified workers* and automates the compliance in between. That gap is the reason Shramik Sathi exists.

3. How it works

The product is delivered as **two portals over one shared backend**:

- **Employer portal** (professional desktop UI) — manage the workforce, post jobs, run payroll, and track statutory compliance (PF/ESI challans, gate passes, registers, committees).
- **Worker portal** (mobile-first PWA) — browse and apply to verified jobs, view earnings & leave, see PF/ESI & compliance status, and report safety issues.

Technology

- **Supabase** — PostgreSQL database + authentication (Google sign-in) + **Row-Level Security (RLS)**, which enforces, at the database level, that each company sees only its own data and each worker sees only their own.
- **Static front-end** (HTML/CSS/JS) hosted on **Vercel**, auto-deployed from GitHub.

How employers and employees are connected (the core loop)

1	A worker registers	creates an employee record (identity / KYC), not yet attached to any company.
2	An employer registers	creates a company record and posts jobs .
3	Worker applies	a job application links that worker to that job.
4	Employer makes an offer; worker confirms	the worker is onboarded : linked to the company, marked Active.
5	Onboarded	the worker now flows into that company's attendance, payroll & compliance — and only now do those features apply to them.

This loop is the backbone of the product — it is exactly the “**employer↔employee connection**” the new work is about making airtight and reliable.

Core data model

Table	Holds	Connected to
companies	Employer organisations	owns employees, jobs
employees	Workers (KYC, wages, status)	belongs to a company (once onboarded)
jobs	Vacancies posted by employers	company → applications
job_applications	A worker applying to a job	links employee ↔ job
attendance / wage_records / leave_records	Day-to-day workforce records	per employee
pf_esi_challans / passes / reports	Compliance, gate passes, grievances	per company / employee

4. What the requested backend work means

My understanding of the four things you asked for:

4.1 Employers & employees fully connected and reliable

- Every employer↔worker relationship is **explicit and integrity-enforced** in the database (foreign keys, no orphaned or dangling records).
- The **onboarding step is atomic** — confirming an offer reliably attaches the worker to the company, sets their status, and updates the job in one safe operation (no half-finished states).
- Statuses stay in sync across applications, employees and jobs so both sides always see the same truth.

4.2 A clean backend that handles traffic without crashing

- **Indexes** on every key/filter column so queries stay fast as data grows.
- **Efficient, single-trip queries** (no N+1), and **RLS policies that don't recursively call each other** (we already hit and fixed one such bug that was returning HTTP 500 on the jobs tables).
- Cross-table permission checks done via safe **helper functions** so they can't loop or leak.
- **Abuse protection** on public forms (leads / issue reports) so they can't be spammed into a crash.
- Supabase's managed connection pooling handles concurrency; the app avoids unbounded queries and uses pagination.

4.3 Compliance dashboard only after onboarding

A worker who has registered but is **not yet hired by any employer** has no payroll, attendance, PF/ESI or compliance data — those only exist once a company onboards them. So:

- Before onboarding (no company linked) → the worker sees an **"awaiting onboarding" / apply-to-jobs** home, **not** the compliance dashboard.
- After an employer onboards them (company linked) → the **full compliance dashboard unlocks** automatically.
- This is enforced both in the UI and by RLS, so it can't be bypassed.

4.4 A unique ID for every worker who registers

- On registration, each worker is auto-assigned a **unique, permanent, human-readable Shramik Sathi ID** (e.g., **SS-2026-000123**).
- It is **portable** — it stays with the worker across employers, and is used on gate passes, payslips, ID cards and the QR badge.
- Generated by the database itself (a sequence) so it is guaranteed unique even when many people register at the same time.

5. How I will implement it (high level)

A detailed plan will follow for your approval; at a high level:

- **A new database migration** that: adds the worker-ID sequence + auto-assignment, enforces the missing foreign-key / integrity constraints, hardens the onboarding so it is fully atomic, and adds any missing indexes.
- **Front-end changes** so the worker home is gated on onboarding status and shows the unique ID on the profile.
- **A reliability pass** over RLS and queries (no recursion, proper indexes, efficient reads), validated by probing the live database.

6. Access & an important constraint

The **anon key** you referenced is already in the project, and I use it to **read and test** the live backend (within the security rules). However — and this matters — the anon key **cannot change the database structure** (tables, security policies, triggers); that is exactly what it is designed to prevent.

To apply backend changes there are two options:

- **(Recommended)** I provide the migration as SQL and you paste-and-run it in the Supabase SQL Editor — the same safe workflow we've used so far.
- You provide a **service-role key** for direct changes — powerful, but it bypasses all security and must **never be shared in chat or committed**; I'd advise against it.

7. What “done” looks like

- A worker registers → instantly gets a unique ID and sees an **onboarding-pending** home (no compliance dashboard yet).
- Worker applies → employer hires → worker confirms → **instantly onboarded**: compliance dashboard unlocks and they appear in the employer's system — one reliable, atomic flow.
- The backend stays fast and **does not crash under concurrent users**; data on both sides is always consistent.

Please confirm this understanding is correct (or correct me). Once you approve, I'll produce the detailed implementation plan and the migration for sign-off before any change is made.